

Introduction to R

Session 07: Iteration and Functions

Álvaro Pérez

Instituto Tecnológico Autónomo de México

Spring 2025

Overview of Today's Session

Iteration functions

Custom functions

Iterating your own functions

Iteration functions

rep()

The function `rep()` lets you repeat the first argument a number of times.

Use `rep()` to create a vector of alternating "A" and "B" values of length 24.

```
1 rep(c("A", "B"), times = 12)
```

If you don't specify what the second argument is, it defaults to `times`, repeating the vector in the first argument that many times. If the second argument is a vector that is the same length as the first argument, each element in the first vector is repeated that many times.

```
1 rep(c("A", "B"), c(11, 3))
```

You can repeat each element of the vector a specified number of times using the `each` argument.

Iteration functions

seq()

The function `seq()` is useful for generating a sequence of numbers with some pattern. Use `seq()` to create a vector of the integers 0 to 10.

```
1 seq(0, 10)
```

You can set the `by` argument to count by numbers other than 1 (the default). Use `seq()` to create a vector of the numbers 0 to 100 by 10s.

```
1 seq(0, 100, by = 10)
```

The argument `length.out` is useful if you know how many steps you want to divide something into. Use `seq()` to create a vector that starts with 0, ends with 100, and has 12 equally spaced steps

Iteration functions

replicate()

You can use the `replicate()` function to run a function `n` times. For example, you can get 3 sets of 5 numbers from a random normal distribution by setting `n` to 3 and `expr` to `rnorm(5)`

```
1 replicate(n = 3, expr = rnorm(5))
```

By default, `replicate()` simplifies your result into a matrix that is easy to convert into a table if your function returns vectors that are the same length. If you'd rather have a list of vectors, set `simplify = FALSE`.

```
1 replicate(n = 3, expr = rnorm(5), simplify = FALSE)
```

Custom functions

Structuring a function

The general structure of a function is as follows:

```
1 function_name <- function(my_args) {  
2   # process the arguments  
3   # return some value  
4 }
```

Here is a very simple function.

```
1 add1 <- function(my_number) {  
2   my_number + 1  
3 }  
4 add1(10)
```

Let's make a function that reports p-values in APA format (with “p = [rounded value]” when $p \geq .001$ and “p < .001” when $p < .001$).

```
1 report_p <- function() {  
2 }
```

Custom a function

Arguments

We need to add one argument, the p-value you want to report.

- ▶ The names you choose for the arguments are private to that argument, so it is not a problem if they conflict with other variables in your script.
- ▶ You put the arguments in the parentheses of `function()` in the order you want them to default (just like the built-in functions you've used before).

```
1 report_p <- function(p) {  
2 }
```

Custom a function

Argument defaults

You can add a default value to any argument. If that argument is skipped, then the function uses the default argument.

- It probably doesn't make sense to run this function without specifying the p-value, but we can add a second argument called `digits` that defaults to 3, so we can round p-values to any number of digits.

```
1 report_p <- function(p, digits = 3) {  
2 }
```

Now we need to write some code inside the function to process the input arguments and turn them into a returned output. Put the output as the last item in function.

Custom a function

Argument defaults

```
1 report_p <- function(p, digits = 3) {  
2   if (p < .001) {  
3     reported = "p<.001"  
4   } else {  
5     roundp <- round(p, digits)  
6     reported = paste("p=", roundp)  
7   }  
8  
9   reported  
10 }
```

You might also see the returned output inside of the return() function. This does the same thing.

Custom a function

Argument defaults

```
1 report_p <- function(p, digits = 3) {  
2   if (p < .001) {  
3     reported = "p < .001"  
4   } else {  
5     roundp <- round(p, digits)  
6     reported = paste("p =", roundp)  
7   }  
8  
9   return(reported)  
10 }
```

When you run the code defining your function, it doesn't output anything, but makes a new object in the Environment tab under Functions. Now you can run the function.

```
1 report_p(0.04869)  
2 report_p(0.0000023)
```

What happens in a function stays in a function. You can change the value of a variable passed to a function, but that won't change the value of the variable outside of the function, even if that variable has the same name as the one in the function.

```
1 reported <- "not_changed"
2
3 # inside this function, reported == "p = 0.002"
4 report_p(0.0023)
5
6 reported # still "not changed"
```

Iterating your own functions

rnorm()

- Create a vector of 20 random numbers drawn from a normal distribution with a mean of 5 and standard deviation of 1 using the `rnorm()` function and store them in the variable `A`.

```
1 A <- rnorm(20, mean = 5, sd = 1)
```

tibble::tibble()

- A tibble is a type of table or data.frame. The function `tibble::tibble()` creates a tibble with a column for each argument. Each argument takes the form `column_name = data_vector`.

```
1 dat <- tibble(  
2   A = rnorm(20, 5, 1),  
3   B = rnorm(20, 5.5, 1)  
4 )
```

Iterating your own functions

t.test()

- ▶ You can run a Welch two-sample t-test by including the two samples you made as the first two arguments to the function `t.test`. You can reference one column of a table by its names using the format `table_name$column_name`

```
1 t.test(dat$A, dat$B)
```

broom::tidy()

- ▶ You can use the function `broom::tidy()` to extract the data from a statistical test in a table format. The example below pipes everything together.

```
1 tibble(  
2   A = rnorm(20, 5, 1),  
3   B = rnorm(20, 5.5, 1)) %>%  
4   gather(group, score, A:B) %>%  
5   t.test(score~group, data = .) %>%  
6   broom::tidy()
```

Iterating your own functions

t_sim()

Custom function

- First, name your function t_sim and wrap the code above in a function with no arguments.

```
1 t_sim <- function() {  
2   tibble(  
3     A = rnorm(20, 5, 1),  
4     B = rnorm(20, 5.5, 1)  
5   ) %>%  
6     gather(group, score, A:B) %>%  
7     t.test(score~group, data = .) %>%  
8     broom::tidy() %>%  
9     pull(p.value)  
10 }  
11 t_sim()
```

Iterating your own functions

t_sim()

Iterate

- ▶ Run the t_sim function 1000 times, assign the resulting p-values to a vector called reps, and check what proportion of p-values are lower than alpha (e.g., .05). This number is the power for this analysis.

```
1 reps <- replicate(1000, t_sim())  
2 alpha <- .05  
3 power <- mean(reps < alpha)  
4 power
```

Iterating your own functions

set.seed

- Use the set.seed function before you run a function that uses random numbers to make sure to get the same random data back each time.

```
1 set.seed(90201)
```

Add arguments

- You can just edit your function each time you want to calculate power for a different sample n, but it is more efficient to build this into your function as an argument.

```
1 t_sim <- function(n = 10, m1=0, sd1=1, m2=0, sd2=1) {
2   tibble(A = rnorm(n, m1, sd1),
3     B = rnorm(n, m2, sd2)) %>%
4     gather(group, score, A:B) %>%
5     t.test(score~group, data = .) %>%
6     broom::tidy() %>%
7     pull(p.value)}
8 # Test your function
```

```
9 t_sim(100)
10 t_sim(100, 0, 1, 0.5, 1)
```